

Context OS – MCP Server Guide

Author: Senior Engineering Team

Project: Context OS – Conversational Analytics Platform

Version: 1.0

Date: June 2026

What This Document Covers

This document explains everything about building, deploying, and understanding MCP Servers for Context OS. It covers what MCP is, why we need two servers, all transport and implementation options with honest tradeoffs, the recommended approach, deployment steps, and the complete flow of what happens when a user sends a query.

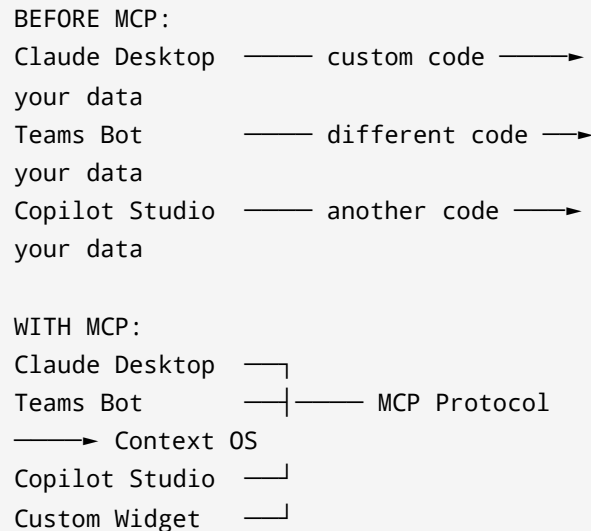
Part 1 – Understanding MCP

What MCP Is

MCP stands for Model Context Protocol. It is an open standard created by Anthropic that defines how AI models communicate with external tools and data sources.

The best analogy is USB-C. Before USB-C, every device had its own port. You needed a different cable for every device. USB-C standardised the connection so any device works with any cable.

MCP does the same thing for AI tools. Before MCP, every AI client needed a custom integration with every tool. With MCP, any client that speaks the protocol connects to any server that speaks the protocol — no custom code.

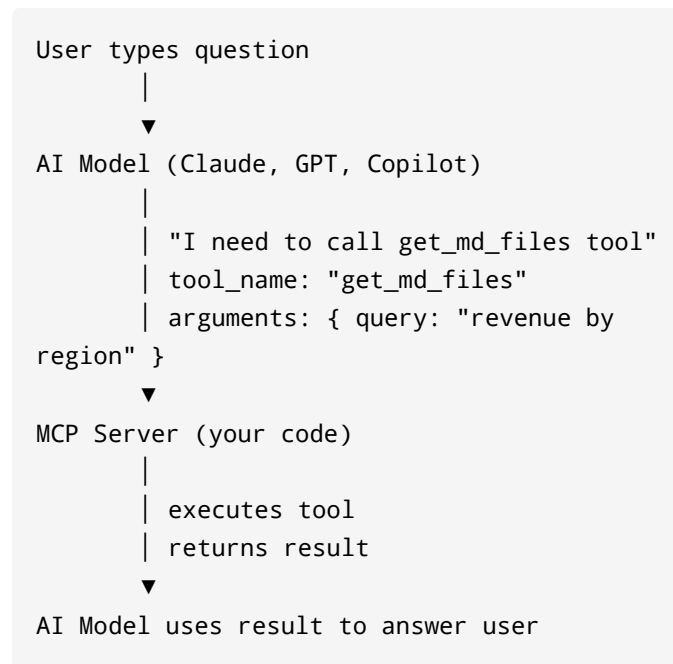


How MCP Works At A High Level

MCP is a client-server protocol built on JSON. The AI model is always the client. Your tools are the server.

The server exposes Tools — named functions with typed inputs and outputs. The AI model calls these tools by name during a

conversation. The server executes and returns results. The AI uses those results to continue the conversation.



Why Context OS Needs MCP

Context OS currently has a FastAPI REST API. Without MCP, only our own frontend can use it. With MCP, the following clients can use Context OS without any custom integration:

- *Claude Desktop (Anthropic)*
- *Microsoft Copilot Studio*
- *Microsoft Teams AI bots*
- *Any custom chat widget*
- *Any future MCP-compatible client*

MCP turns Context OS from a private API into a universal analytics tool any AI client can use.

Part 2 — Our Two MCP Server Architecture

Why Two Servers

Most systems build one MCP server. We build two. This is a deliberate architectural decision.

The key insight is that our system has two very different jobs:

Job 1 — Intelligence (stateful, complex, needs memory)

- *Know who the user is*
- *Remember what they asked before*
- *Find the right context for their question*
- *Manage session state*
- *Run background agents*

Job 2 — Execution (stateless, simple, just runs SQL)

- *Receive SQL*
- *Route to correct database*
- *Execute and return results*
- *No memory needed between calls*

Mixing these two jobs in one server causes problems.

Intelligence work is CPU-heavy and needs session memory.

Execution work is IO-heavy and needs to scale with user volume. Splitting them means each can scale independently and fail independently.

Server 1 – Orchestration

```
Name:      Context OS Orchestration
Port:      8001
State:     Stateful (knows about sessions,
           users, context)
Job:       Intelligence layer
Connects:  Redis, Postgres, pgvector,
           context_docs/ files
```

Server 1 is the brain. Every user query starts here. It figures out what the user is asking, finds the right context, manages the conversation memory, and runs agents when needed.

What it does NOT do – execute SQL. That is Server 2's job.

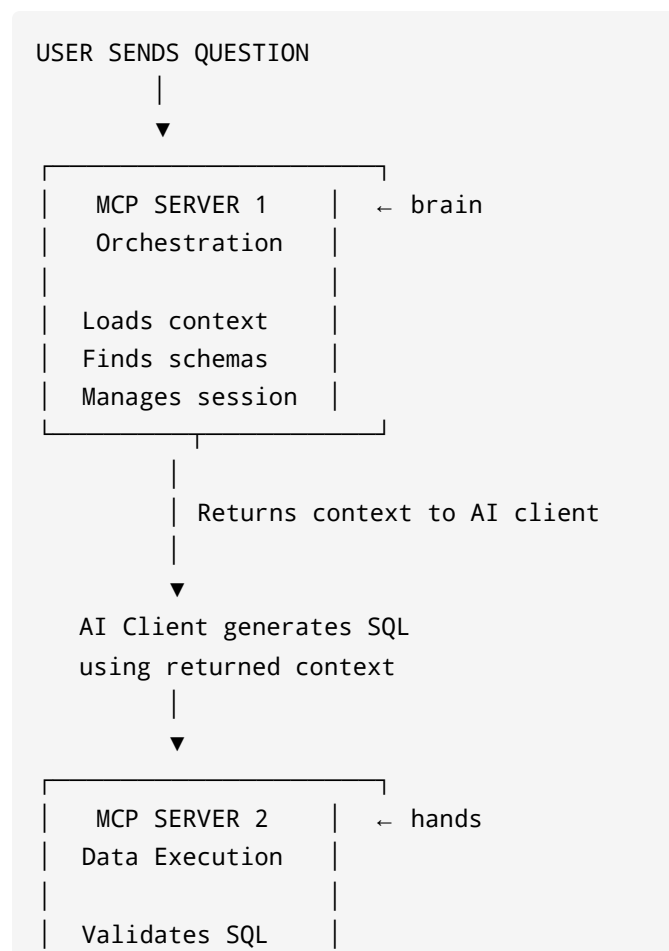
Server 2 – Data Execution

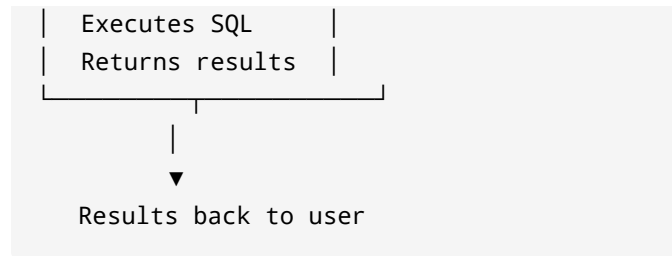
```
Name:      Context OS Data Execution
Port:      8002
State:     Stateless (no memory between
           calls)
Job:       Execution layer only
Connects:  Postgres (connector registry),
           data warehouses
```

Server 2 is the hands. It receives SQL and a target, executes it, returns results. It knows nothing about sessions or context. Every call is independent.

Adding a new data source (Snowflake, Fabric, Synapse) only changes Server 2. Server 1 never needs to change.

How They Work Together





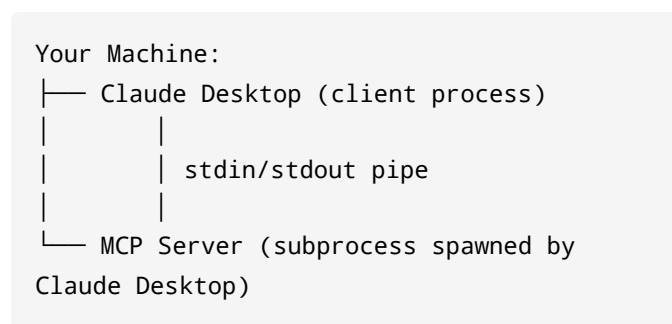
Part 3 — Transport Options

Transport is how the client and server communicate. This is the most important decision because it determines who can connect to your servers.

Option A — stdio

What it is: Server runs as a subprocess. Client communicates through stdin and stdout pipes. Like two processes talking through a pipe on the same machine.

How it looks:



Honest assessment:

✓ <i>Simplest setup</i>	<i>No network config needed</i>
✓ <i>Works locally</i>	<i>Zero ports to open</i>
✓ <i>Claude Desktop native</i>	<i>Supported out of the box</i>
✗ <i>Local only</i>	<i>Cannot serve network clients</i>
✗ <i>No scaling</i>	<i>One server per client process</i>
✗ <i>No Teams</i>	<i>Cannot be used by Teams or Copilot Studio</i>
✗ <i>No monitoring</i>	<i>Hard to observe what is happening</i>

Use when: Personal local use with Claude Desktop only. Not for production.

Option B — HTTP with Server-Sent Events (SSE) ✓

Recommended

What it is: Server runs as an HTTP service. Client connects via HTTP. Server streams responses using SSE — a one-way streaming protocol built on HTTP that sends events as text over a persistent connection.

How it looks:

Network:

```
├─ Claude Desktop — HTTP —> MCP
Server (port 8001)
├─ Teams Bot      — HTTP —> MCP
Server (port 8001)
├─ Copilot Studio — HTTP —> MCP
Server (port 8001)
└─ Custom Widget  — HTTP —> MCP
Server (port 8001)
```

Honest assessment :

✓ Works over network	Any client anywhere can connect
✓ Teams + Copilot Studio	Enterprise client support
✓ Horizontal scaling	Put a load balancer in front
✓ Standard HTTP	Works through corporate firewalls
✓ Easy monitoring	Datadog, Prometheus, all standard tools
✓ Health checks	Standard HTTP health endpoints
✓ Deploy anywhere	Containers, cloud, Kubernetes
✗ More setup	Need to manage server deployment

*Use when: Any production deployment. Any enterprise use.
Multiple clients.*

Option C – WebSocket

What it is: Persistent bidirectional connection. Both client and server can send messages at any time without the other asking first.

Honest assessment:

 Lower latency	<i>Persistent connection, no reconnect overhead</i>
 True bidirectional	<i>Server can push to client unprompted</i>
 Firewall issues	<i>Many enterprise firewalls block WebSocket</i>
 Complex reconnection	<i>Must handle drops and reconnects</i>
 Overkill	<i>Tool calling is request/response, not streaming</i>

Use when: Real-time streaming scenarios. Not needed for standard tool calling patterns. We do not use this.

Transport Decision

We use HTTP/SSE for both servers.

Reason: We need Teams, Copilot Studio, and Claude Desktop all working. Only HTTP/SSE supports all three in production.

For Claude Desktop local development:
Configure it to point to the HTTP servers running locally. No studio needed.

Part 4 — Python Implementation Options

Three ways to build MCP servers in Python.

Option A — Official MCP Python SDK Only

Install the official Anthropic MCP library and build purely with it.

```
pip install mcp
```

How it works: The SDK provides a `Server` class with decorators for registering tools. You define tools and handlers. The SDK handles all protocol details.

Honest assessment:

✓ Protocol guaranteed	Anthropic maintains compliance
✓ Type safe	Pydantic models built in
✓ Official support	Anthropic maintains it
✓ Less boilerplate	Protocol handled for you
✗ Less control	HTTP layer is opinionated
✗ Newer library	Fewer community examples
✗ Hard to add middleware	Auth, logging harder to add

Option B – Pure FastAPI (Manual Protocol)

Build MCP protocol yourself on top of FastAPI. No MCP SDK.

How it works: You write the SSE endpoint, parse MCP JSON messages yourself, route to handlers, format responses manually.

Honest assessment :

✓ Full control	Every byte under your control
✓ Familiar	Just FastAPI, nothing new to learn

✓ Easy middleware	Auth, logging trivial in FastAPI
✗ Protocol risk	Any mistake breaks all clients
✗ Maintenance burden	Protocol updates = manual work
✗ More code	300+ lines of boilerplate

Option C — FastAPI + MCP SDK Combined ✓

Recommended

Use the MCP SDK for protocol handling. Use FastAPI for everything else.

How it works: MCP SDK manages tool registration and protocol. FastAPI manages HTTP lifecycle, health checks, auth middleware, and CORS. They mount together — FastAPI serves the HTTP routes that the MCP SDK uses for SSE.

Honest assessment:

✓ Protocol correct	MCP SDK handles compliance
✓ Full HTTP control	FastAPI handles middleware, auth
✓ Health checks easy	Standard FastAPI routes
✓ Monitoring easy	FastAPI middleware for metrics

✓ Familiar patterns	Our existing FastAPI knowledge applies
✓ Shared code	Reuses our config, database, connectors
✗ Two frameworks	Both need to be understood
✗ Integration complexity	Mounting them together takes care

This is our choice. Best of both worlds.

Part 5 — What Each Server Does In Detail

MCP Server 1 — The 8 Tools

Tool 1: `get_md_files`

The most important tool. Called on every user query.

- *Loads session state from Redis*
- *Extracts structured intent from query using fast Haiku model*
- *Searches pgvector for semantically relevant tables*
- *Loads corresponding MD files from context_docs/schema/*
- *Searches knowledge vectors for relevant business context*

- *Returns combined context string the LLM uses to generate SQL*
- *Cost: ~\$0.0002 per call (Haiku) + 15ms pgvector search*

Tool 2: `run_agent`

Triggers one of the four background agents on demand.

- `schema_crawler` — *crawl and index schemas from a connector*
- `sentinel` — *check for schema changes, update pgvector*
- `harvester` — *ingest documents from SharePoint/S3/Confluence*
- `compiler` — *generate session MD file and store in Postgres*

Tool 3: `refresh_graph`

Rebuilds the knowledge graph from all current MD files.

Called after new schemas are crawled or documents ingested.

- *Reads all files in context_docs/*
- *Extracts entities and relationships*
- *Rebuilds in-memory graph*

Tool 4: `get_session_context`

Loads the current conversation state from Redis.

- *Active tables being referenced*
- *Current filters (year, region, etc.)*
- *Last user intent*
- *Token count for this session*

- *Last 5 conversation turns*

Tool 5: update_session

Updates Redis after each turn completes.

- *Adds new tables to active set*
- *Merges new filters*
- *Appends turn to rolling turns list*
- *Updates token count*

Tool 6: get_token_budget

Checks how much of the session token budget remains.

- *Returns used, remaining, percentage*
- *Returns alert flag if over 90%*
- *Returns hard_stop flag if at 100%*

Tool 7: optimize_prompt

Compresses the context string to fit within token budget.

Called when get_md_files returns more context than fits.

- *Summarises older session turns*
- *Removes redundant schema sections*
- *Returns optimised context that fits budget*

Tool 8: get_user_profile

Loads user information from Postgres.

- *Organisation context*
- *User role and permissions*
- *Custom token budget*

- *Preferences*
-

MCP Server 2 – The 6 Tools

Tool 1: `execute_query`

The core execution tool. Receives SQL, executes it, returns results.

- *Gets connector from registry by ID*
- *Refreshes auth token if expired*
- *Enforces read-only (no writes ever)*
- *Enforces row cap (default 10,000)*
- *Returns rows, columns, execution time*
- *Completely stateless — no session state involved*

Tool 2: `validate_sql`

Three-layer SQL validation before execution.

- *Layer 1: Syntax check using sqlglot (dialect-aware)*
- *Layer 2: Schema grounding (every table and column must exist)*
- *Layer 3: Safety guard (blocks DROP, DELETE, UPDATE, INSERT)*
- *Returns valid/invalid with reason and suggestion*

Tool 3: `test_connection`

Tests if a connector can reach its data source.

- *Called before a session starts*

- *Called by health monitoring*
- *Returns status (healthy/degraded/unhealthy) + latency*

Tool 4: `list_connectors`

Lists all registered connectors for an organisation.

- *Returns name, type, status for each*
- *Used by UI connector management*

Tool 5: `get_schema_sample`

Returns sample rows from a specific table.

- *Helps LLM understand what the data actually looks like*
- *Runs `SELECT * FROM table LIMIT 5`*
- *Returns columns + sample rows*

Tool 6: `get_connector_dialect`

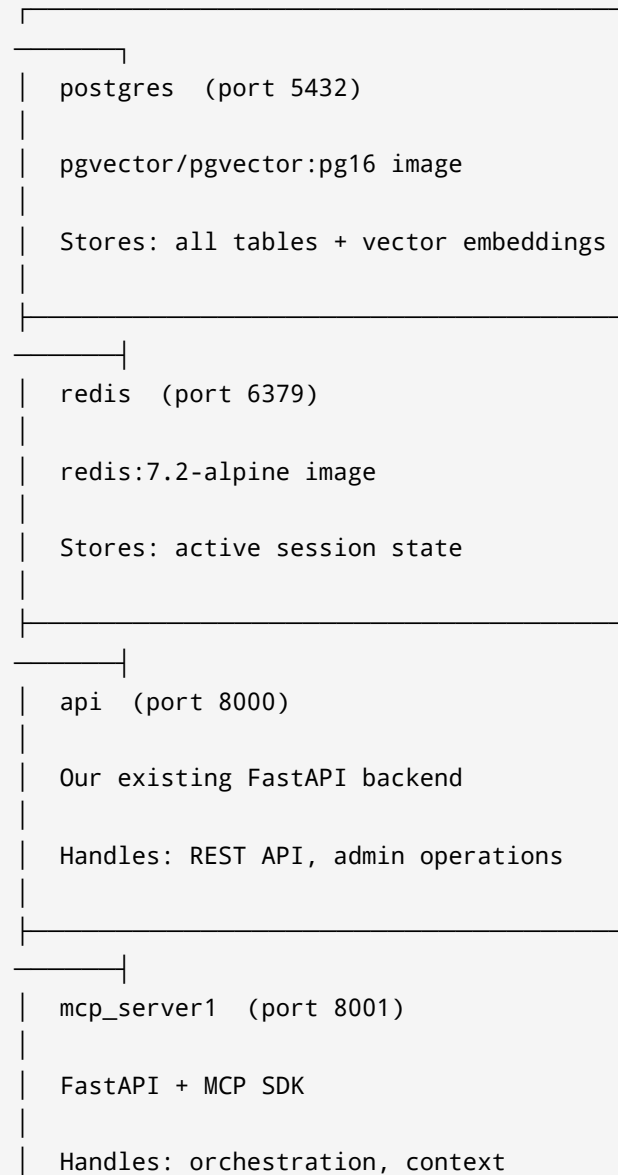
Returns the SQL dialect for a connector.

- *Databricks → `sparksql`*
- *Fabric → `tsql`*
- *Snowflake → `snowflake`*
- *PostgreSQL → `postgresql`*
- *LLM uses this to know which SQL flavor to generate*

Part 6 — Deployment

What Runs Where

docker-compose brings up 5 services:



```
|
|-----|
|
| mcp_server2 (port 8002)
|
| FastAPI + MCP SDK
|
| Handles: SQL execution only
|
|-----|
|
```

Step by Step Deployment

Step 1 – Prerequisites

```
# Ensure you have:
docker --version          # Docker 24+
docker-compose --version  # Docker Compose
2.0+
python --version          # Python 3.11+
```

Step 2 – Environment Setup

```
# Copy env template
cp .env.example .env

# Generate required keys
python -c "
import secrets
from cryptography.fernet import Fernet
print('SECRET_KEY=' +
```

```

secrets.token_hex(32))
print('JWT_SECRET_KEY=' +
secrets.token_hex(32))
print('ENCRYPTION_KEY=' +
Fernet.generate_key().decode())
"

# Fill in .env with:
# POSTGRES_PASSWORD=yourpassword
# ANTHROPIC_API_KEY=sk-ant-...
# OPENAI_API_KEY=sk-...
# DATABRICKS_HOST=https://adb-
xxx.azuredatabricks.net
#
DATABRICKS_HTTP_PATH=/sql/1.0/warehouses/xx
x
# DATABRICKS_PAT_TOKEN=dapi-xxx

```

Step 3 – Start Infrastructure

```

# Start Postgres + Redis
docker-compose up -d postgres redis

# Wait for healthy
docker-compose ps

# Enable pgvector and create tables
python scripts/create_tables.py

```

Step 4 – Bootstrap Crawl

This runs once. Crawls Databricks schemas and stores in pgvector.

```
# Run bootstrap crawl
python test_crawl.py

# Expected output:
# ✅ Auth success
# ✅ Crawl complete: 47 tables
# ✅ Embedding complete: 47 tables embedded
# ✅ Vectors stored in schema_embeddings
# ✅ MD files saved to context_docs/schema/
```

Step 5 – Start All Services

```
# Start everything
docker-compose up -d

# Verify all running
docker-compose ps

# Check health of MCP servers
curl http://localhost:8001/health
curl http://localhost:8002/health
```

Step 6 – Add pgvector Index

Run this after bootstrap crawl. Makes vector search 10x faster.

```
-- Connect to Postgres and run:
CREATE INDEX ON schema_embeddings
USING ivfflat (embedding vector_cosine_ops)
WITH (lists = 100);
```

```
CREATE INDEX ON knowledge_embeddings
USING ivfflat (embedding vector_cosine_ops)
WITH (lists = 100);
```

Step 7 – Connect Claude Desktop

Find config file:

- **Mac:** ~/Library/Application Support/Claude/claude_desktop_config.json
- **Windows:** %APPDATA%\Claude\claude_desktop_config.json

Add:

```
{
  "mcpServers": {
    "context-os-orchestration": {
      "url":
"http://localhost:8001/mcp1/sse",
      "transport": "sse"
    },
    "context-os-execution": {
      "url":
"http://localhost:8002/mcp2/sse",
      "transport": "sse"
    }
  }
}
```

Restart Claude Desktop. Tools will appear automatically.

Production Scaling

Small team (under 50 users):

mcp_server1: 1 instance, 2 workers

mcp_server2: 1 instance, 4 workers

Cost: minimal

Medium scale (50-500 users):

mcp_server1: 2 instances behind load balancer

mcp_server2: 4 instances behind load balancer

Add: Redis with persistence

Add: Postgres connection pooler (PgBouncer)

Large scale (500+ users):

mcp_server1: 4+ instances

mcp_server2: 8+ instances (auto-scale on CPU)

Add: Dedicated Postgres for vectors

Add: Redis Cluster

Add: CDN for context_docs/ files

Part 7 — Complete Query Flow

This is what happens from the moment a user types a question to when they see an answer.

Scenario

```
User: John at TechCorp
Client: Claude Desktop
Question: "Show me total revenue by region
for Q1 2026"
Data in: Databricks warehouse
```

Phase 1 – Context Retrieval (Server 1)

What happens: Claude Desktop sees the user's question and automatically calls `get_md_files` on Server 1.

```
Claude Desktop calls: get_md_files
Input: {
  query: "Show me total revenue by region
for Q1 2026",
  session_id: "john-session-abc123",
  user_id: "john@techcorp.com"
}
```

Inside Server 1 – `get_md_files`:

```
1. Load session from Redis
Key: session:john-session-abc123:state
Returns: {
  active_tables: [],
  filters: {},
  connector_id: "databricks-techcorp"
}
```

Time: 1ms

2. Extract intent (Haiku model – fast and cheap)

Input: "Show me total revenue by region for Q1 2026"

Output: {
 metrics: ["revenue"],
 filters: {region: "all regions"},
 time_period: "Q1 2026",
 source_hint: "databricks",
 confidence: 0.94
}

Time: 180ms | Cost: \$0.0002

3. Search pgvector for relevant tables

Embed query → 1536-number vector

Search schema_embeddings table

Results:

main.sales.orders similarity: 0.94

main.sales.customers similarity: 0.87

main.finance.invoices similarity: 0.71

Time: 15ms

4. Load MD files from disk

context_docs/schema/databricks_main_sales_orders.md 

context_docs/schema/databricks_main_sales_customers.md 

Time: 2ms

5. Search knowledge pgvector

Returns: "Revenue = gross_sales -
returns..."

"Q1 = January, February, March
(fiscal)"

Time: 10ms

6. Build context string

Schema Context

Table: main.sales.orders (MANAGED)

Description: All customer orders since
2020

Column	Type	Description
-----	-----	-----

order_id	BIGINT	Primary key
customer_id	BIGINT	FK to
customers		
order_date	DATE	When order
placed		
total_amount	DECIMAL	Order value
in USD		

Table: main.sales.customers (MANAGED)

...columns...

Business Knowledge

Revenue = gross_sales minus returns and
discounts

Q1 = January, February, March fiscal

```
year
```

```
Total time: ~210ms
```

Server 1 returns to Claude Desktop:

```
{
  context_string: "## Schema Context\n...",
  tables_identified: [
    "main.sales.orders",
    "main.sales.customers"
  ],
  intent: {
    metrics: ["revenue"],
    time_period: "Q1 2026",
    confidence: 0.94
  },
  token_estimate: 620
}
```

Phase 2 – Token Budget Check (Server 1)

Claude Desktop calls `get_token_budget` to check if the user has budget remaining before calling the expensive LLM.

```
Server 1 returns:
{
  used: 4250,
  budget: 100000,
  remaining: 95750,
  pct_used: 4.25%,
```

```
    alert: false
  }

  Budget is fine. Continue.
```

Phase 3 – SQL Generation (AI Client, No Server Involved)

The AI client (Claude Desktop) now has all the context it needs. It constructs a system prompt and generates SQL itself. No MCP server involved in this step.

```
System prompt Claude Desktop sends to
Claude Sonnet:

You are an expert SparkSQL analyst for
Databricks.
Generate a single valid SparkSQL SELECT
query.

SCHEMA CONTEXT:
[content of orders MD file]
[content of customers MD file]

BUSINESS KNOWLEDGE:
Revenue = gross_sales - returns...
Q1 = January, February, March...

SESSION CONTEXT:
Active tables: none yet
Recent turns: none (first turn)
```

User question: "Show me total revenue by region for Q1 2026"

Claude Sonnet generates:

```
SELECT
  c.region,
  SUM(o.total_amount) AS revenue
FROM main.sales.orders o
JOIN main.sales.customers c
  ON o.customer_id = c.customer_id
WHERE YEAR(o.order_date) = 2026
      AND QUARTER(o.order_date) = 1
      AND o.status != 'cancelled'
GROUP BY c.region
ORDER BY revenue DESC
```

Time: 800ms | Cost: ~\$0.012

Phase 4 – SQL Validation (Server 2)

Before executing anything, Claude Desktop calls
`validate_sql` ***on Server 2.***

```
Claude Desktop calls: validate_sql
Input: {
  sql: "SELECT c.region,
SUM(o.total_amount)...",
```

```
connector_id: "databricks-techcorp"
}
```

Server 2 runs three checks:

Check 1 – Safety (fastest, runs first):

First word: SELECT

Not in write keywords

(DROP/DELETE/UPDATE...)

✓ Safe

Check 2 – Syntax:

sqlglot parses with SparkSQL dialect

No parse errors

✓ Valid syntax

Check 3 – Schema grounding:

main.sales.orders → found in
schema_embeddings ✓

main.sales.customers → found in
schema_embeddings ✓

o.total_amount → found in
column_names ✓

c.region → found in
column_names ✓

✓ All tables and columns verified

Server 2 returns: {

valid: true,

reason: "All checks passed"

}

Time: 10ms

Phase 5 – SQL Execution (Server 2)

Claude Desktop calls `execute_query` on *Server 2*.

```
Claude Desktop calls: execute_query
Input: {
  sql: "SELECT c.region,
SUM(o.total_amount)...",
  connector_id: "databricks-techcorp",
  session_id: "john-session-abc123"
}

Server 2 does:

1. Get connector from registry
  registry.get("databricks-techcorp")
  → DatabricksConnector instance

2. Check token freshness
  PAT still valid → continue
  (or: SP token expiring → refresh →
  continue)

3. Execute on SQL Warehouse
  SQL hits Databricks warehouse
  Runs against actual Delta tables

4. Returns:
{
  rows: [
    { region: "APAC", revenue: 4200000 },
    { region: "NA", revenue: 3800000 },
```

```
{ region: "EMEA", revenue: 2100000 }
],
columns: ["region", "revenue"],
row_count: 3,
execution_ms: 1240,
truncated: false,
error: null
}

Time: 1240ms
```

Phase 6 – Session Update (Server 1)

Claude Desktop calls `update_session` on Server 1 with what was learned from this turn.

```
Claude Desktop calls: update_session
Input: {
  session_id: "john-session-abc123",
  entities: {
    region: "all",
    time_period: "Q1 2026",
    year: 2026,
    quarter: 1
  },
  tables_used: [
    "main.sales.orders",
    "main.sales.customers"
  ],
  filters: { year: 2026, quarter: 1 }
}
```

Server 1 updates Redis:

```
session:john-session-abc123:state = {  
  active_tables: [  
    "main.sales.orders",  
    "main.sales.customers"  
  ],  
  filters: { year: 2026, quarter: 1 },  
  last_intent: "revenue by region",  
  token_count: 5490,  
  turn_count: 1  
}
```

Appends to session:john-session-
abc123:turns:

"User: Show me total revenue by region
for Q1 2026"

"Assistant: APAC \$4.2M, NA \$3.8M, EMEA
\$2.1M"

Time: 3ms

Phase 7 — Answer Presented To User

***Claude Desktop combines the SQL results with an LLM-
generated interpretation and shows the user:***

Answer:

"APAC led Q1 2026 revenue at \$4.2M,
followed

by NA at \$3.8M and EMEA at \$2.1M. Total Q1 revenue was \$10.1M across all regions."

SQL (expandable):

```
SELECT c.region, SUM(o.total_amount) AS  
revenue...
```

Results table:

region	revenue
APAC	4,200,000
NA	3,800,000
EMEA	2,100,000

Token usage: 5,490 / 100,000 (5.5%)

Complete Timing Summary

```
Phase 1: get_md_files (Server 1)  
210ms  
Phase 2: get_token_budget (Server 1)  
1ms  
Phase 3: SQL generation (AI model)  
800ms  
Phase 4: validate_sql (Server 2)  
10ms  
Phase 5: execute_query (Server 2)  
1240ms  
Phase 6: update_session (Server 1)
```

3ms

Total end to end:
~2.3s

What Happens On The Next Turn

Because session was updated, the next question benefits from context memory.

User asks: "Break it down by month"

Server 1 loads session from Redis:
 active_tables: [orders, customers] ← remembered
 filters: {year: 2026, quarter: 1} ← remembered

LLM system prompt now includes:
 "Active tables: orders, customers
 Current filters: year=2026, quarter=Q1
 Last query was about: revenue by region"

LLM understands "it" means the Q1 revenue
LLM understands "by month" means add
MONTH() to GROUP BY
LLM generates correct SQL without being
told the filters again

This is what session memory enables.

Part 8 – Connecting Different Clients

Claude Desktop (Local)

```
// ~/Library/Application
Support/Claude/claude_desktop_config.json
{
  "mcpServers": {
    "context-os-orchestration": {
      "url":
"http://localhost:8001/mcp1/sse",
      "transport": "sse"
    },
    "context-os-execution": {
      "url":
"http://localhost:8002/mcp2/sse",
      "transport": "sse"
    }
  }
}
```

Restart Claude Desktop. Tools appear automatically in the tool menu.

Microsoft Teams Bot

Teams bot connects to Server 1 to get context, generates SQL using Azure OpenAI, then calls Server 2 to execute.

Connection: HTTP POST to `/mcp1/sse` and `/mcp2/sse`
Auth: API key in header `X-API-Key`

Microsoft Copilot Studio

Add a connector in Copilot Studio:

```
Connector type: HTTP
Base URL: https://your-domain.com
Authentication: API Key
Header name: X-API-Key
```

Copilot Studio auto-discovers tools from the MCP
`list_tools` endpoint.

Custom React Frontend

Uses the MCP TypeScript SDK to connect to both servers. The frontend calls Server 1 tools to get context, uses that context with a chat completion API, then calls Server 2 tools to execute the generated SQL.

Part 9 — Key Design Principles

Server 1 is stateful. Server 2 is stateless.

Never put session state in Server 2. Never put SQL execution in Server 1.

Adding a new data source only touches Server 2.

Create a new connector implementing BaseConnector. Register it in the registry. Server 1 never changes.

Every tool returns JSON.

Tools return structured JSON, not plain text. Clients parse it and decide how to present it.

Failures are graceful.

If Server 1 knowledge search fails, fall back to prompt_text from pgvector.

If Server 2 execution fails, return the error in the response, do not crash.

No tool should ever return an unhandled exception to the client.

The AI client generates SQL. The servers provide context and execution.

Neither server generates SQL. That is the LLM's job. Server 1 gives context. Server 2 runs what the LLM produces.

Part 10 — Production Readiness Checklist

Before going live:

Infrastructure:

- ☐ pgvector/pgvector:pg16 Docker image (not plain postgres)
- ☐ Bootstrap crawl completed (schemas in pgvector)
- ☐ pgvector IVFFlat index created after

crawl

- ☐ Redis persistence enabled
- ☐ context_docs/ volume mounted to both servers
- ☐ All env vars populated

Security:

- ☐ API key authentication on MCP endpoints
- ☐ HTTPS configured (TLS certificate)
- ☐ CORS restricted to known origins
- ☐ Credentials encrypted (ENCRYPTION_KEY set)
- ☐ .env not in git (in .gitignore)

Performance:

- ☐ Server 2 has more workers than Server 1
- ☐ pgvector indexes created
- ☐ Redis maxmemory configured
- ☐ Connection pools tuned

Monitoring:

- ☐ /health endpoints returning 200 on both servers
- ☐ Structured logs shipping to log aggregator
- ☐ Alerts on tool error rate above 1%
- ☐ Alerts on p95 latency above 5 seconds
- ☐ Token budget monitoring dashboard live

Summary Reference

<i>Topic</i>	<i>Decision</i>
<i>Transport</i>	<i>HTTP/SSE</i>
<i>Python implementation</i>	<i>FastAPI + MCP Python SDK</i>
<i>Number of servers</i>	<i>2 (orchestration + execution)</i>
<i>Server 1 port</i>	<i>8001</i>
<i>Server 2 port</i>	<i>8002</i>
<i>Server 1 tools</i>	<i>8 tools</i>
<i>Server 2 tools</i>	<i>6 tools</i>
<i>Vector search</i>	<i>pgvector inside Postgres</i>
<i>Session memory</i>	<i>Redis</i>
<i>Context format</i>	<i>MD files + pgvector prompt_text</i>
<i>Target query time</i>	<i>Under 3 seconds end to end</i>

Build Server 2 first — it is simpler and stateless, ideal for testing execution.

Then build Server 1 — it depends on all the intelligence components being ready.